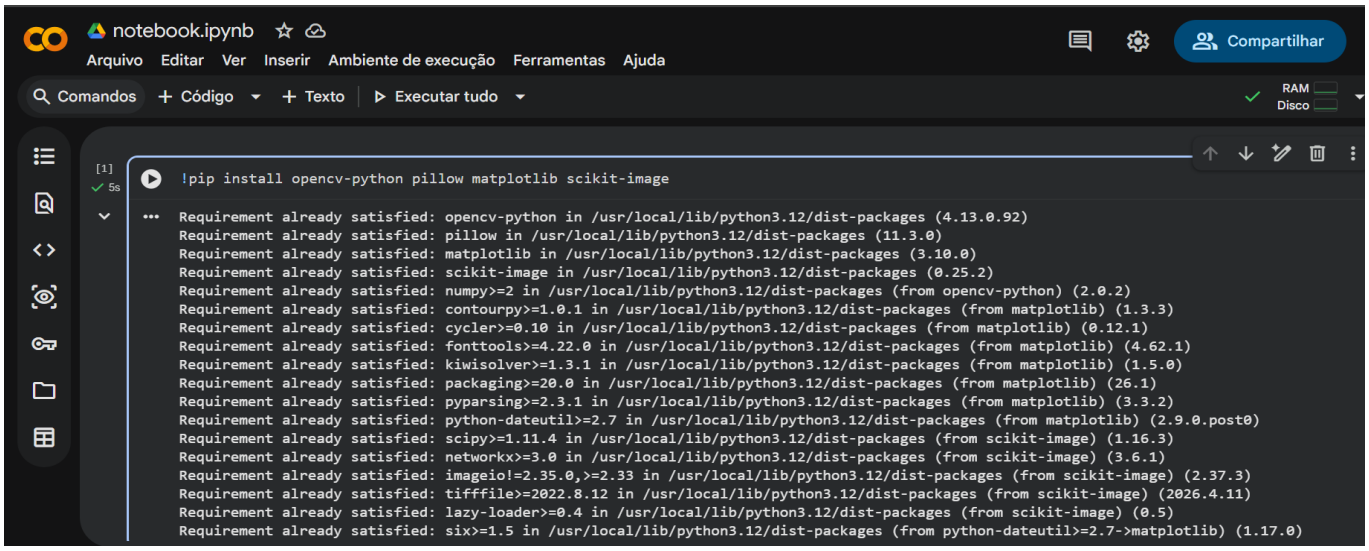
Roteiro Oficina - Manipulação de imagens em Python para apoiar o ensino de Pensamento Computacional no Ensino Médio

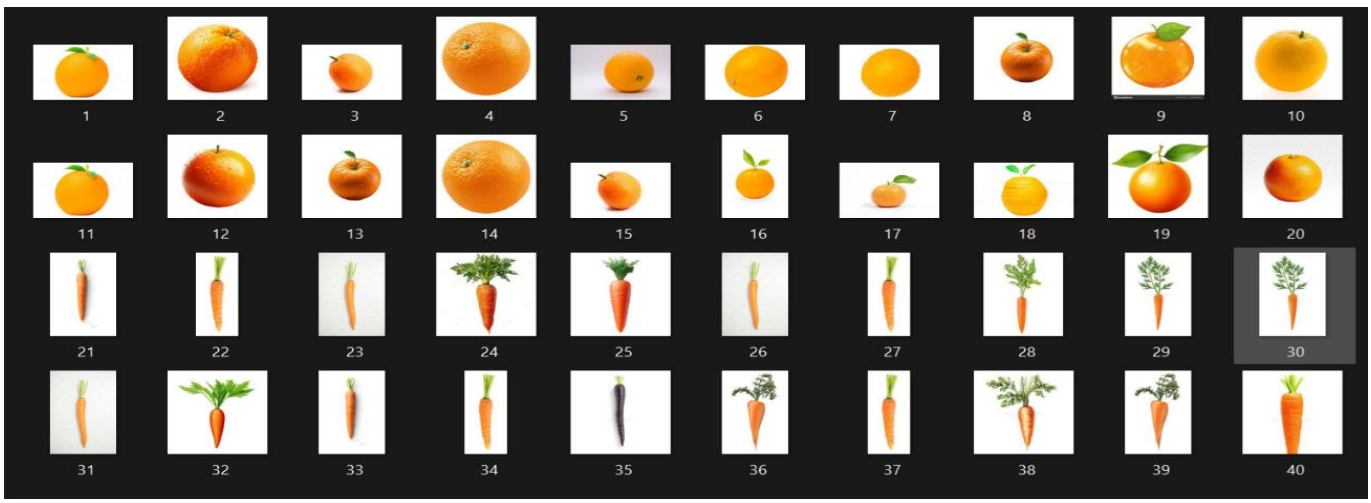
Ambiente Google Colaboratory (Colab)



Importação Biblioteca

```
!pip install opencv-python pillow matplotlib scikit-image
!pip install pillow
import cv2
import PIL
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from PIL import Image
import matplotlib.pyplot as plt
get_ipython().system('pip install pillow')
from matplotlib import image
from matplotlib import pyplot
from matplotlib import pyplot as plt
```

Imagens usadas no Processamento



Carregar Imagem no Driver

```
from google.colab.patches import cv2_imshow
import numpy as np
drive.mount('/content/drive')


imagemOriginal1 = Image.open("/content/drive/MyDrive/Grupo1/1.jpg")
imagemOriginal2 = Image.open("/content/drive/MyDrive/Grupo1/2.jpg")
imagemOriginal3 = Image.open("/content/drive/MyDrive/Grupo1/3.jpg")
imagemOriginal4 = Image.open("/content/drive/MyDrive/Grupo1/4.jpg")
(…)
imagemOriginal40 = Image.open("/content/drive/MyDrive/Grupo1/40.jpg")
```

Plotar Imagem

```
imgplot = plt.imshow(imagemOriginal1)
plt.show()
```

Imprimir formato, tamanho (largura, altura) e modo (RGB, etc)

```
print(f"Formato: { imagemOriginal1.format}")
print(f"Tamanho (Largura x Altura): { imagemOriginal1.size}")
print(f"Modo: { imagemOriginal1.mode}")
```

Convert Pillow Image to NumPy array

```
numpy_image1 = np.array(imagemOriginal1) # Convert Pillow Image to NumPy array
imagemCinza1 = cv2.cvtColor(numpy_image1, cv2.COLOR_BGR2GRAY)
numpy_image2 = np.array(imagemOriginal2) # Convert Pillow Image to NumPy array
imagemCinza2 = cv2.cvtColor(numpy_image2, cv2.COLOR_BGR2GRAY)
numpy_image3 = np.array(imagemOriginal3) # Convert Pillow Image to NumPy array
imagemCinza3 = cv2.cvtColor(numpy_image3, cv2.COLOR_BGR2GRAY)
numpy_image4 = np.array(imagemOriginal4) # Convert Pillow Image to NumPy array
(…)
imagemCinza40 = cv2.cvtColor(numpy_image40, cv2.COLOR_BGR2GRAY)
numpy_image40 = np.array(imagemOriginal40) # Convert Pillow Image to NumPy array
```

Plotar Imagem

```
imgplot = plt.imshow(imagemCinza1)
plt.show()
```

Binarização da Imagem

```
Imagem_Binary1 = cv2.threshold(imagemCinza1, 170, 255, cv2.THRESH_BINARY)
Imagem_Binary2 = cv2.threshold(imagemCinza2, 170, 255, cv2.THRESH_BINARY)
Imagem_Binary3 = cv2.threshold(imagemCinza3, 170, 255, cv2.THRESH_BINARY)
(…)
Imagem_Binary40 = cv2.threshold(imagemCinza40, 170, 255, cv2.THRESH_BINARY)
```

Plotar Imagem

```
Imagem_Binary1
plt.imshow(Imagem_Binary1[1], cmap='gray')
```

Suavização de Imagem - Média

```
Imagem_Media1 = cv2.blur(Imagem_Binary1[1], (5,5))
Imagem_Media2 = cv2.blur(Imagem_Binary2[1], (5,5))
Imagem_Media3 = cv2.blur(Imagem_Binary3[1], (5,5))
(…)
Imagem_Media40 = cv2.blur(Imagem_Binary40[1], (5,5))
```

Plotar imagem

```
plt.imshow(Imagem_Media1, cmap='gray')
```

Suavização de Imagem - Mediana

```
Imagem_Mediana1 =cv2.medianBlur(Imagem_Media1,5)
Imagem_Mediana2 =cv2.medianBlur(Imagem_Media2,5)
Imagem_Mediana3 =cv2.medianBlur(Imagem_Media3,5)
(...)
Imagem_Mediana40 =cv2.medianBlur(Imagem_Media40,5)
```

Plotar Imagem

```
plt.imshow(Imagem_Mediana1, cmap='gray')
```

Suavização de Imagem - Glaussiano

```
Imagem_Gaussiano1 =cv2.GaussianBlur(Imagem_Mediana1, (5,5) , 0)
Imagem_Gaussiano2 =cv2.GaussianBlur(Imagem_Mediana2, (5,5) , 0)
Imagem_Gaussiano3 =cv2.GaussianBlur(Imagem_Mediana3, (5,5) , 0)
(...)
Imagem_Gaussiano40 =cv2.GaussianBlur(Imagem_Mediana40, (5,5) , 0)
```

Plotar Imagem

```
plt.imshow(Imagem_Gaussiano1, cmap='gray')
```

Segmentação - Binarização Invertida

```
Imagem_BinaryInvertida1
=cv2.threshold(Imagem_Gaussiano1,80,255,cv2.THRESH_BINARY_INV)
Imagem_BinaryInvertida2
=cv2.threshold(Imagem_Gaussiano2,80,255,cv2.THRESH_BINARY_INV)
Imagem_BinaryInvertida3
=cv2.threshold(Imagem_Gaussiano3,80,255,cv2.THRESH_BINARY_INV)
(…)
```

```
Imagem_BinaryInvertida40
=cv2.threshold(Imagem_Gaussiano40,80,255,cv2.THRESH_BINARY_INV)
```

Plotar Imagem

```
plt.imshow(Imagem_BinaryInvertida1[1], cmap='gray')
```

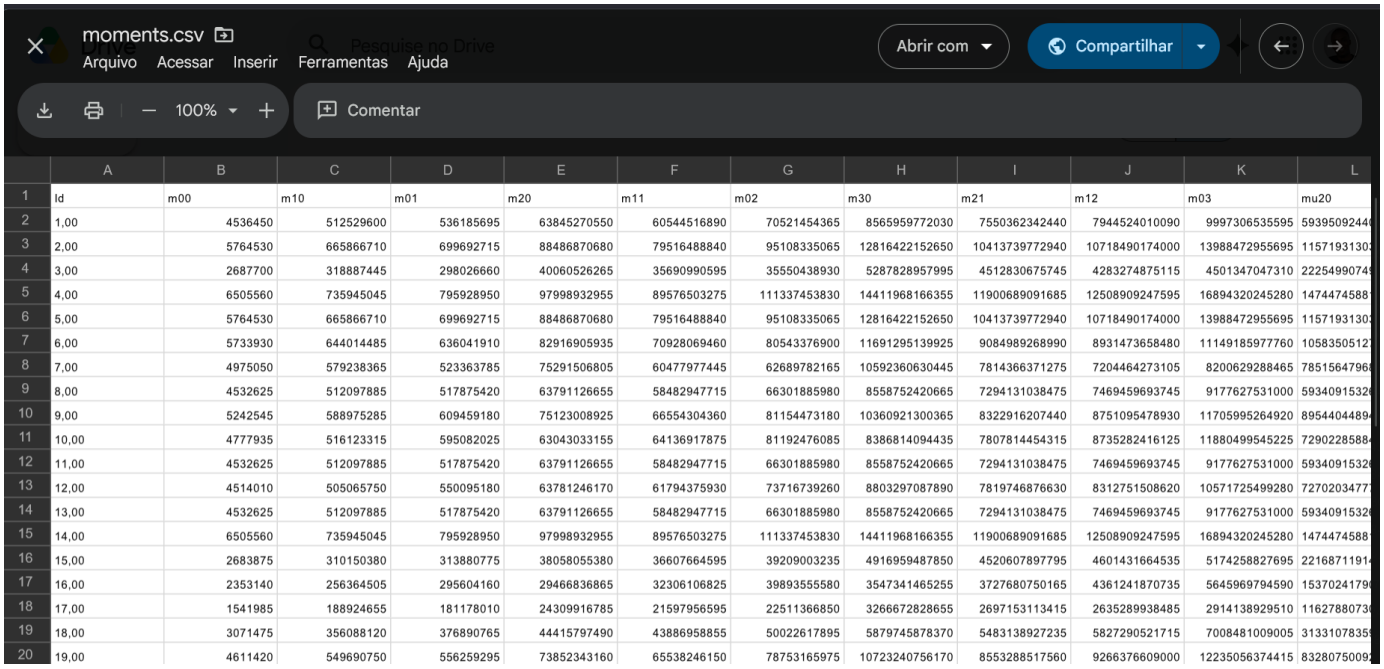
Segmentação - Extrair os dados Moments

```
moments1 = cv2.moments(Imagem_BinaryInvertida1[1])
moments2 = cv2.moments(Imagem_BinaryInvertida2[1])
moments3 = cv2.moments(Imagem_BinaryInvertida3[1])
(...)
moments40 = cv2.moments(Imagem_BinaryInvertida40[1])
```

Segmentação - Extrair os dados Humoments

```
humoments1 = cv2.HuMoments(moments1)
humoments2 = cv2.HuMoments(moments2)
humoments3 = cv2.HuMoments(moments3)
(...)
humoments40 = cv2.HuMoments(moments40)
```

Planilha: Moments.csv



	A	B	C	D	E	F	G	H	I	J	K	L
1	Id	m00	m10	m01	m20	m11	m02	m30	m21	m12	m03	mu20
2	1,00	4536450	512529600	536185695	63845270550	60544516890	70521454365	8565959772030	7550362342440	7944524010090	9997306535595	59395092446
3	2,00	5764530	665866710	699692715	88486870680	79516488840	95108335065	12816422152650	10413739772940	10718490174000	13988472955695	11571931303
4	3,00	2687700	318887445	298026660	40060526265	35690990595	35550438930	5287828957995	4512830675745	4283274875115	4501347047310	22254990749
5	4,00	6505560	735945045	795928950	97998932955	89576503275	111337453830	14411968166355	11900689091685	12508909247595	16894320245280	14744745881
6	5,00	5764530	665866710	699692715	88486870680	79516488840	95108335065	12816422152650	10413739772940	10718490174000	13988472955695	11571931303
7	6,00	5733930	644014485	636041910	82916905935	70928069460	80543376900	11691295139925	9084989268990	8931473658480	11149185977760	10583505121
8	7,00	4975050	579238365	523363785	75291506805	60477977445	62689782165	10592360630445	7814366371275	7204464273105	8200629288465	78515647968
9	8,00	4532625	512097885	517875420	63791126655	58482947715	66301885980	8558752420665	7294131038475	7469459693745	9177627531000	59340915326
10	9,00	5242545	588975285	609459180	75123008925	66554304360	81154473180	10360921300365	8322916207440	8751095478930	11705995264920	89544044894
11	10,00	4777935	516123315	595082025	63043033155	64136917875	81192476085	8386814094435	7807814454315	8735282416125	11880499545225	72902285884
12	11,00	4532625	512097885	517875420	63791126655	58482947715	66301885980	8558752420665	7294131038475	7469459693745	9177627531000	59340915326
13	12,00	4514010	505065750	550095180	63781246170	61794375930	73716739260	8803297087890	7819746876630	8312751508620	10571725499280	72702034777
14	13,00	4532625	512097885	517875420	63791126655	58482947715	66301885980	8558752420665	7294131038475	7469459693745	9177627531000	59340915326
15	14,00	6505560	735945045	795928950	97998932955	89576503275	111337453830	14411968166355	11900689091685	12508909247595	16894320245280	14744745881
16	15,00	2683875	310150380	313880775	38058055380	36607664595	39209003235	4916959487850	4520607897795	4601431664535	5174258827695	22168711914
17	16,00	2353140	256364505	295604160	29466836865	32306106825	39893555580	3547341465255	3727680750165	4361241870735	5645969794590	15370241790
18	17,00	1541985	188924655	181178010	24309916785	21597956595	22511366850	3266672828655	2697153113415	2635289938485	2914138929510	11627880730
19	18,00	3071475	356088120	376890765	44415797490	43886958855	50022617895	5879745878370	5483138927235	5827290521715	7008481009005	31331078359
20	19,00	4611420	549690750	556259295	73852343160	65538246150	78753165975	10723240756170	8553288517560	9266376609000	12235056374415	83280750093

Carregar Planilha no Drive

```
data = pd.read_csv('/content/drive/MyDrive/Grupo1/moments.csv',
on_bad_lines='skip', delimiter=';')
moments=pd.DataFrame(data)
# List of columns identified as objects that should be numeric (from
moments.info() and error traceback)
numeric_cols = ['mu20', 'mu11', 'mu02', 'mu30', 'mu21', 'mu12', 'mu03', 'nu20',
'nu11', 'nu30', 'nu21', 'nu12', 'nu03']
for col in numeric_cols:
    if col in moments.columns:
        moments[col] = moments[col].astype(str).str.replace(',', '.',
regex=False).str.replace('--', '-', regex=False).astype(float)
moments.head()
```

Classificação - Aplicação do Algoritmo de Rede Neural

```
moments.info()
plt.figure (figsize= (8,4))
sns.countplot(x= 'tipo_imagem', data = moments)
sns.set_style('whitegrid')
sns.FacetGrid(moments, hue= 'tipo_imagem', height = 6).map(plt.scatter, 'm00',
'm30').add_legend()
plt.show()
sns.set_style('whitegrid')
sns.FacetGrid(moments, hue="tipo_imagem", height = 6).map(plt.scatter, 'mu02',
'mu30').add_legend()
plt.show()
sns.set_style('whitegrid')
sns.FacetGrid(moments, hue="tipo_imagem", height = 6).map(plt.scatter, 'nu02',
'nu30').add_legend()
plt.show()
from sklearn.neural_network import MLPClassifier
moments=moments.values
X = moments[0:, 1:25].astype(float)
Y = moments[0:, 25].astype(object)
print(X.shape)
print(Y.shape)
from sklearn import model_selection
X_train, X_test, Y_train, Y_test = model_selection.train_test_split(X, Y,
test_size=0.3,random_state=0)
print(X_train.shape)
print(Y_test.shape)
model=MLPClassifier(activation='relu', hidden_layer_sizes=(3),
random_state=0,max_iter=20000, verbose=True)
model.fit(X_train, Y_train)
predictions = model.predict(X_test)
Y_test
from sklearn import metrics
metrics.accuracy_score(predictions, Y_test)
prediction = model.predict(X_test)
from sklearn import metrics
metrics.accuracy_score(prediction, Y_test)
```